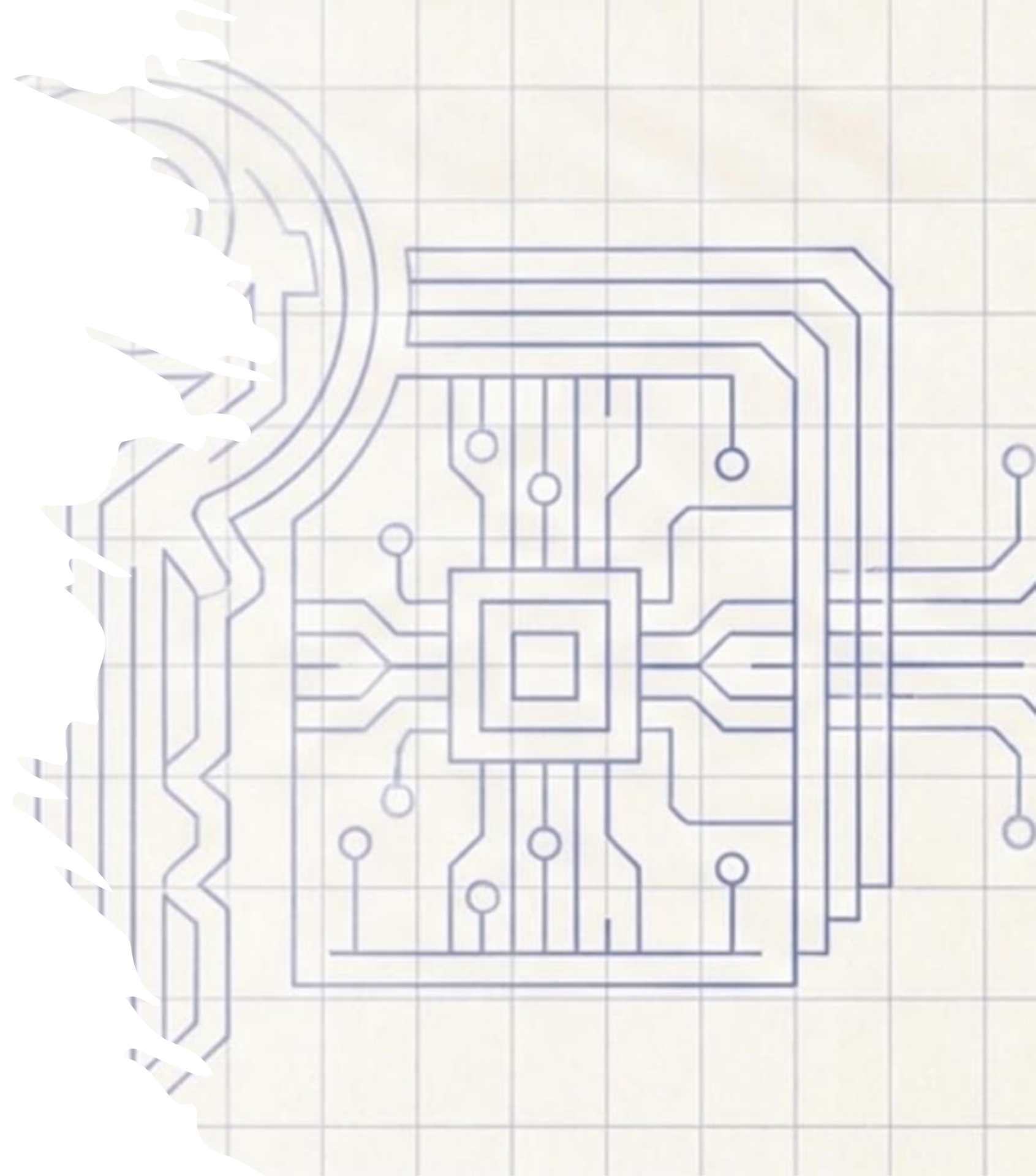


Post – Quantum Token (PQ—ERC20) A Quantum – Resistant Smart Contract Implementation

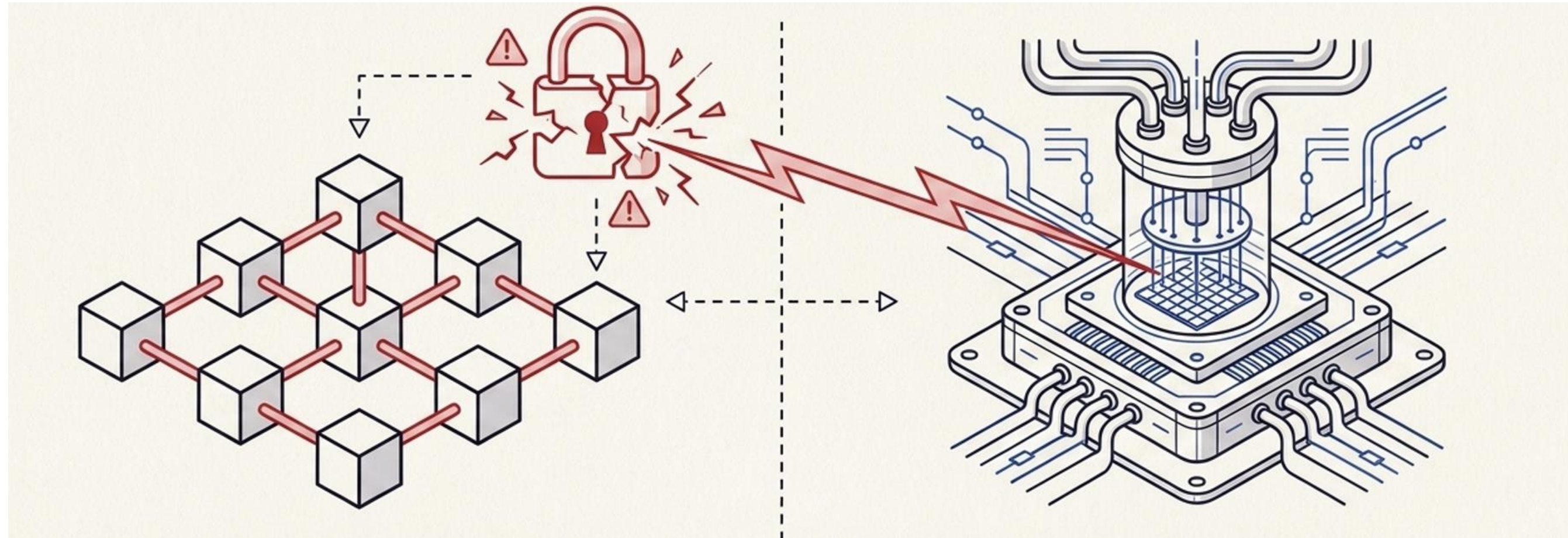
Student: Elif Kuş / 210304003

Course: COMP4052 Introduction to Blockchain and Distributed
Ledger Technology

Branch: 210304003-elif-kus



The Threat: Quantum Computing vs Blockchain



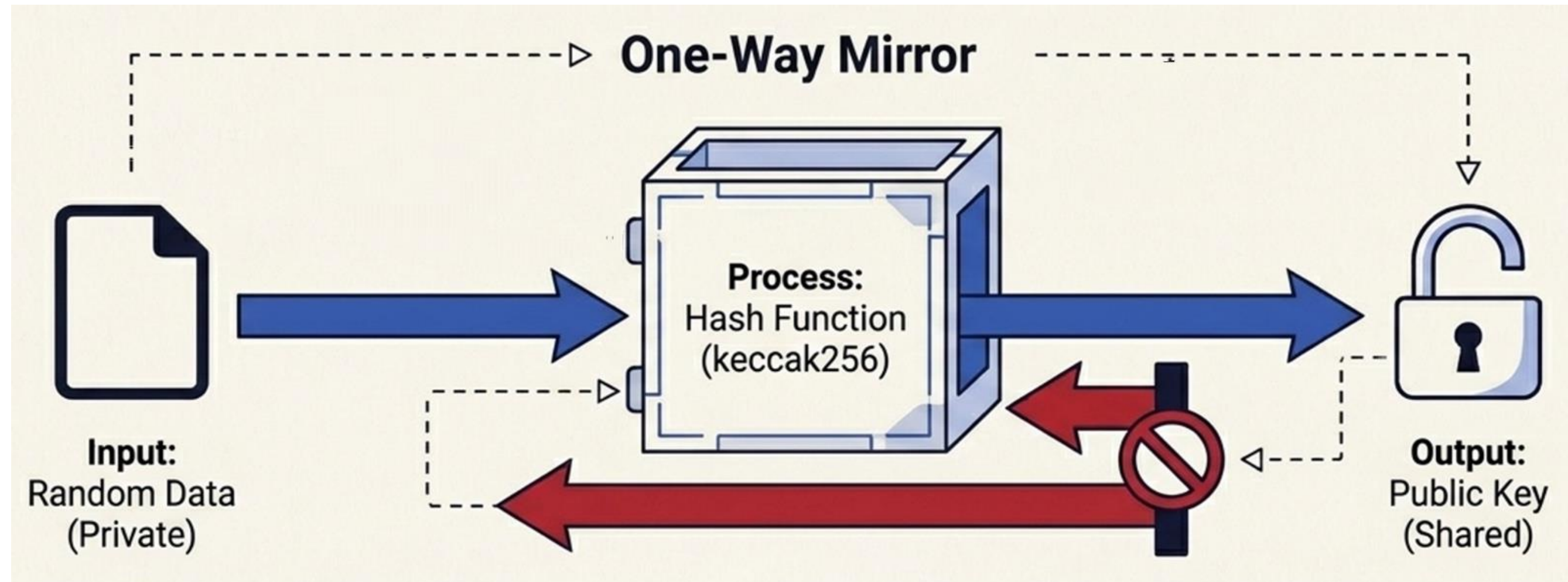
The Standard Protocol:

Current blockchains rely on algorithms like Elliptic Curve Cryptography (ECDSA) for privacy.

The Future Problem:

Quantum computers will mathematically break these algorithms, destroying network privacy.

The Solution: Lamport One – Time Signatures



Hash-Based Cryptography:
Instead of complex elliptic curves, it uses simple, secure hashing.

One-Way Street:
It is easy to hash a private key to make a public key. It is impossible to reverse it.

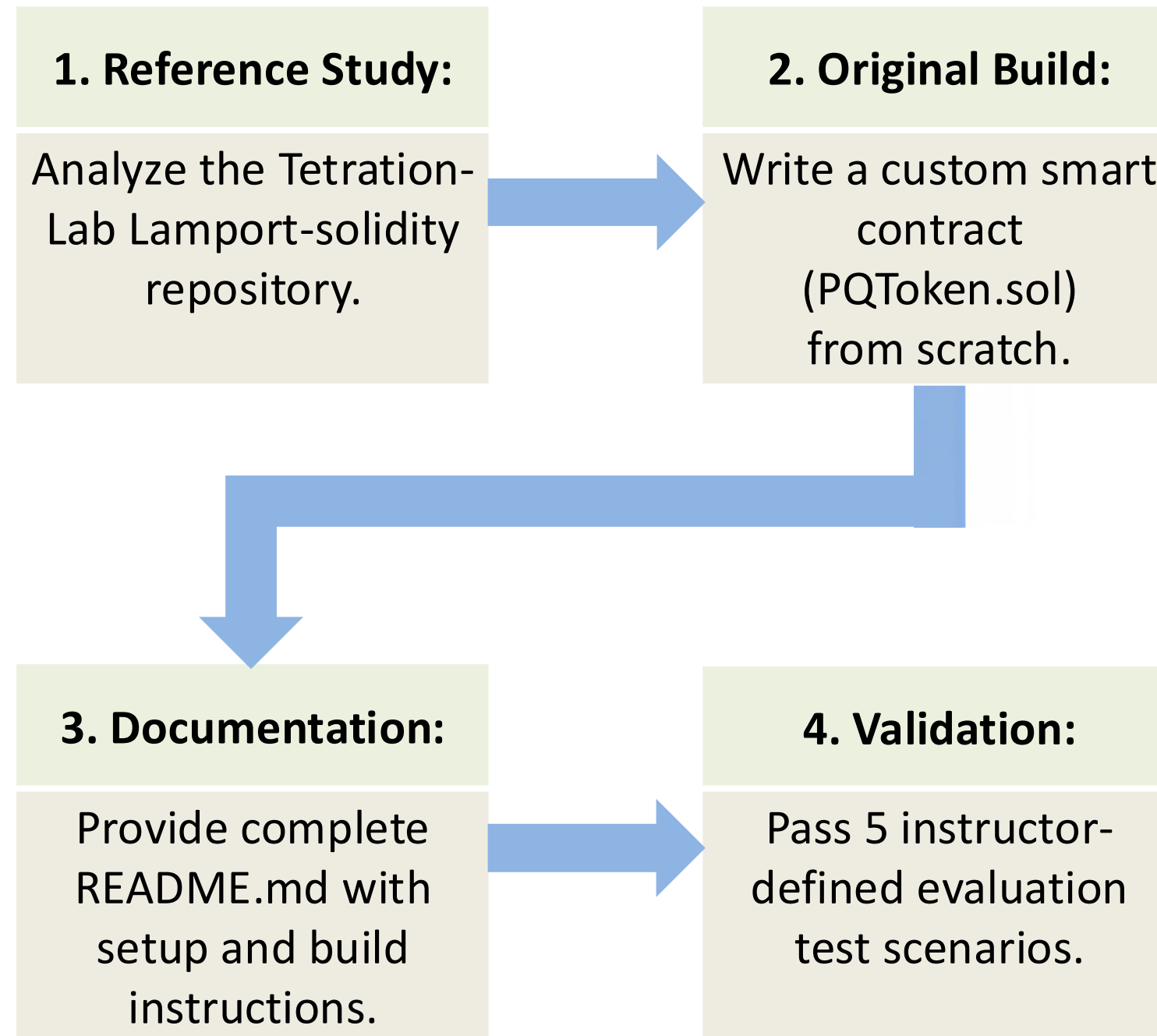
Quantum Safe:
Quantum computers cannot efficiently reverse strong hash functions.

Cryptography Comparison: ECDSA vs Lamport OTS

	Standard ECDSA	Lamport OTS
Quantum Resistance	Vulnerable	Secure
Math Foundation	Elliptic Curves	Hash Functions
Key Reusability	Unlimited	One-Time Use Only
Signature Size	Very Small (65 bytes)	Very Large (Many Hashes)

Conclusion:
Lamport is mathematically secure against quantum attacks but requires an innovative **smart contract** architecture to manage “One-Time” keys on the blockchain.

The COMP4052 Mission Framework



Project Architecture & Tech Stack

Off-Chain Layer (Client Side)

Tools: Node.js, Hardhat, Ethers.js , Chai

Action: Generating massive random keys and signs messages locally.



Transaction Payload
(Message Hash + Lamport Signature)

On-Chain Layer (Blockchain)

Tools: Solidity, EVM

Action: Verifies hashes, updates token balances, and burns used keys

Visualizing the Math: Key Generation



Step 1: Private Key

Generate 256 pairs of 32-byte random numbers.

```
ethers.randomBytes(32)
```

Step 2: Hashing Process

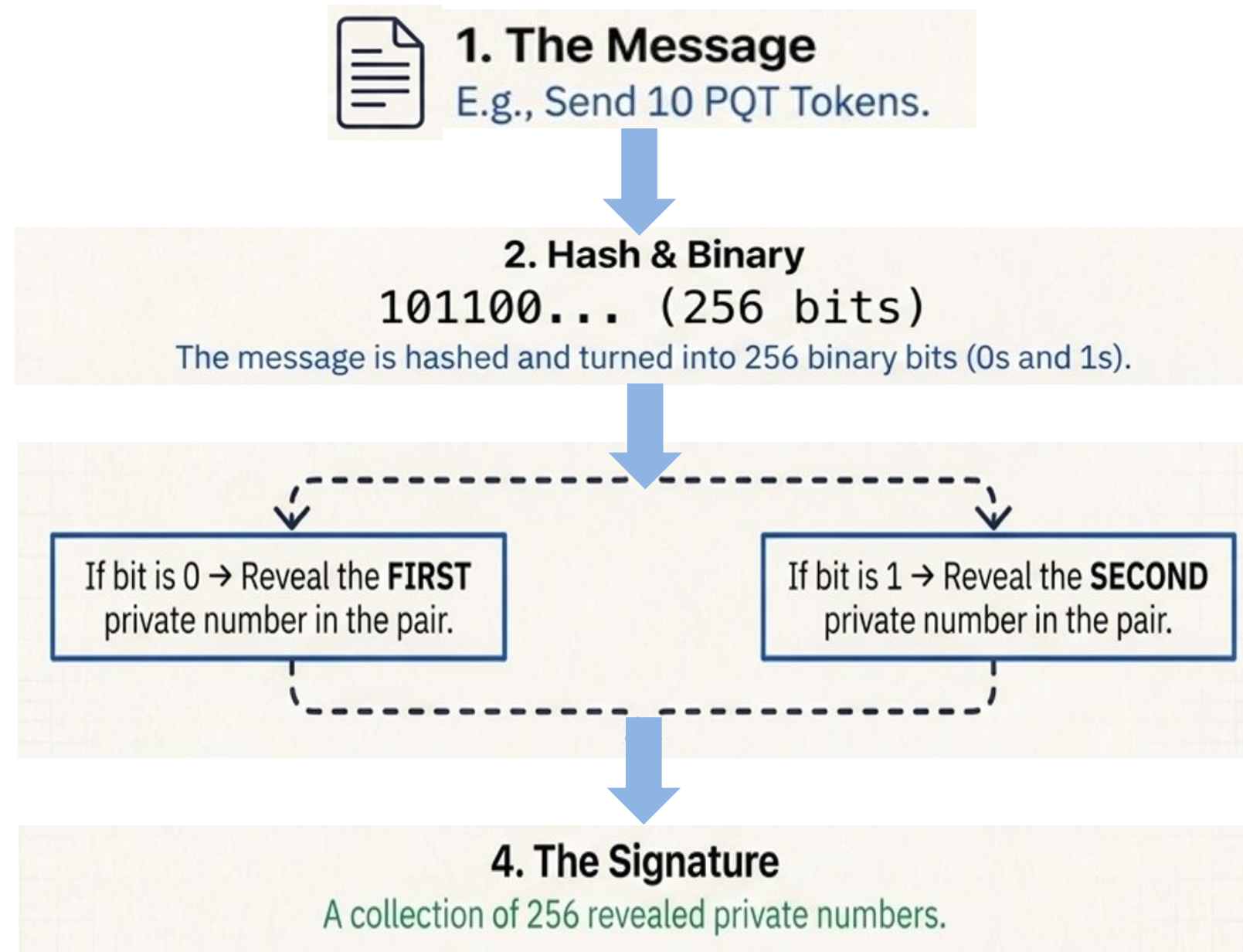
Apply one-way hash to every single number.

```
ethers.keccak(256)
```

Step 3: Public Key

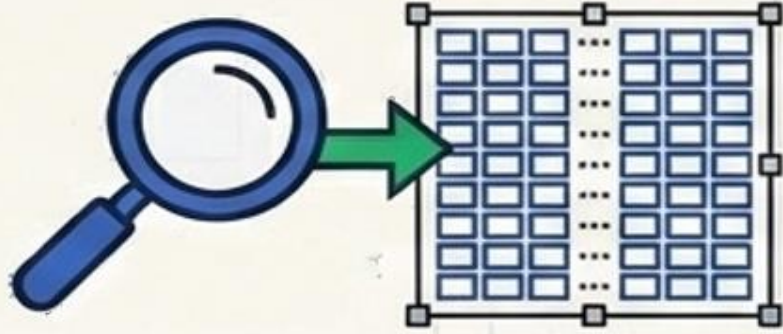
The resulting 256 pairs of hashes saved on the blockchain.

Visualizing the Math: Signing a Transaction



Security Note: Half of the private key is permanently revealed. It can never be used again.

On-Chain Logic: The pqTransfer Function



Step 1: Verification (On-Chain)

The Smart contract hashes the 256 revealed signature numbers and checks if they perfectly match the stored **Public Key**.



Database Node A

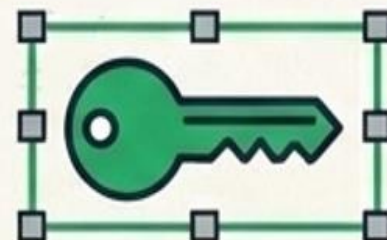
Database Node B

Step 2: Execute Transfer & State Update

If the hashes match, the ERC20 token balances are updated. A **Nonce** is updated to prevent replay attacks.



Old 'Public Key'



New 'Public Key'

Step 3: Key Renewal (The Burn)

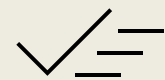
Because '**Lamport**' is "**One-Time**", the old public key is burned. The user provides a **NEW 'Public Key'** during the transfer for their next transaction.

Proving Original Work: Reference vs Implementation

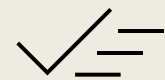
Feature Dimension	Tetration-Lab (Reference)	PQ-ERC20 (Implementation)
Project Type	General Math Library	Functional Token Contract
Token Standard	None	ERC20 – like Token Balances
Key Management	Manual Verification	Automatic One-Time Key Renewal
Security	Basic Math Check	Nonce-Based Replay Protection

The reference project demonstrated how to compute the math. This original project built a usable financial token (PQ-ERC20) utilizing the math.

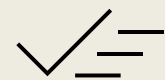
Validation: Passing the 5 Evaluation Scenarios



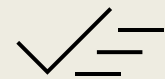
Test 1: Successful off-chain key generation



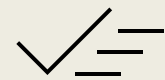
Test 2: Valid signature successfully verified on-chain



Test 3: Token balances updated correctly after pqTransfer



Test 4: Invalid signature rejected by the smart contract



Test 5: Old public key successfully replaced with new key

Synthesis: Future-Proofing Decentralization

The Problem:

Quantum computing threatens the mathematical foundation of current blockchain privacy.



The Solution:

Hash-based cryptography, utilizing the **Lamport OTS** algorithm, provides a mathematically proven shield.



The Proof:

The **PQ-ERC20 token** demonstrates that quantum-resistant smart contracts can be built, deployed, and executed today.